

A* Algorithm for Maze Problem - Viva Preparation

This document contains important viva preparation notes for the A* Algorithm implemented for the Maze/Grid Path Finding problem using Java and ArrayList.

1. Important Concepts

Concept	Explanation
A* Algorithm	A heuristic search algorithm used to find shortest path.
$g(n)$	Actual cost from start node.
$h(n)$	Estimated cost to goal node.
$f(n)$	Total cost = $g(n) + h(n)$.
Heuristic Used	Manhattan Distance.
Open List	Stores nodes waiting to be explored.
Visited Array	Prevents revisiting nodes.

2. Important Formula

$$f(n) = g(n) + h(n)$$

$g(n)$ = actual distance from start

$h(n)$ = estimated distance to goal

3. Functions Used in Code

Function	Purpose
$f()$	Calculates total cost.
getMin()	Finds node having minimum $f(n)$.
astar()	Main A* Algorithm logic.
main()	Creates maze and calls algorithm.

4. Working of Algorithm

1. Add starting node into open list.
2. Find node with minimum $f(n)$.
3. Remove it from open list.
4. Check if goal reached.
5. Explore neighbors in 4 directions.
6. Add valid neighbors into open list.
7. Continue until goal is found.

5. Important Viva Questions

- What is A* Algorithm?

- What is heuristic function?
- What is $f(n)$?
- Difference between BFS and A*?
- Which heuristic is used in this program?
- Why ArrayList is used?
- Why visited array is needed?
- What is Manhattan Distance?
- Why getMin() function is required?
- What happens if no path exists?

6. Short Viva Answers

Question	Answer
What is A*?	A heuristic path finding algorithm.
What is heuristic?	Estimated distance to goal.
Why ArrayList?	To store open nodes.
Which heuristic?	Manhattan Distance.
What is $f(n)$?	$f(n) = g(n) + h(n)$.

7. Time Complexity

Worst Case Time Complexity: $O(b^d)$

Where:

b = branching factor

d = depth of solution